

Vertex Capability System Specification

Version: 1.0

Status: Draft

1. Objetivo

O Capability System é o subsistema do Vertex responsável por permitir que plugins exponham funcionalidades reutilizáveis e que outros plugins as descubram e utilizem sem depender de implementações específicas.

Seu objetivo é fornecer um mecanismo de comunicação baseado em contratos, reduzindo o acoplamento entre plugins e permitindo que múltiplas implementações coexistam.

Uma capability representa o que um plugin é capaz de fazer, e não como essa funcionalidade é implementada.

2. Responsabilidades

O Capability System é responsável por:

- registrar capabilities;
- remover capabilities;
- descobrir capabilities disponíveis;
- resolver múltiplas implementações;
- invocar capabilities;
- validar permissões;
- validar versões;
- gerenciar metadados;
- monitorar uso;
- isolar falhas durante invocações.

3. Arquitetura

Plugin

↓

Capability API

↓

Capability System

- |— Registry
- |— Discovery Service
- |— Resolver
- |— Invocation Engine
- |— Version Manager
- |— Permission Validator
- |— Metadata Manager
- |— Metrics

Toda comunicação passa pelo Capability System.

Plugins nunca chamam diretamente outro plugin.

4. Componentes

Capability Registry

Mantém todas as capabilities registradas.

Cada registro contém:

- ID;
- nome;
- versão;
- plugin proprietário;
- permissões;
- prioridade;
- metadados.

Discovery Service

Permite localizar capabilities.

Suporta pesquisas por:

- ID;
- categoria;
- versão;
- tags;
- tipo.

Capability Resolver

Seleciona a implementação adequada quando existem múltiplos provedores.

Critérios possíveis:

- prioridade;
- versão;
- políticas do sistema;
- preferência do usuário.

Invocation Engine

Executa chamadas para capabilities.

É responsável por:

- localizar o provedor;
- encaminhar parâmetros;
- retornar resultados;
- tratar falhas.

Version Manager

Gerencia compatibilidade entre diferentes versões de capabilities.

Permission Validator

Verifica se o plugin possui autorização para utilizar determinada capability.

Metadata Manager

Armazena informações descritivas sobre cada capability.

Metrics Collector

Coleta estatísticas de utilização.

5. Fluxos

Registro

Plugin



Manifest



Capability Registry



Capability Disponível

Descoberta

Plugin



Discovery Service



Capability Encontrada

Invocação

Plugin



Capability API



Resolver

↓

Invocation Engine

↓

Plugin Provedor

↓

Resultado

Remoção

Plugin

↓

Desregistro

↓

Registry Atualizado

6. APIs

Registro

- registerCapability()
- unregisterCapability()

Descoberta

- discover()
- findById()
- findByCategory()
- findByTag()

Consulta

- metadata()
- provider()
- version()

Invocação

- invoke()
- invokeAsync()

Compatibilidade

- supportedVersions()
- compatible()

7. Estados

Uma capability pode assumir os seguintes estados:

Registered

↓

Available

↓

Busy

↓

Unavailable

↓

Deprecated

↓

Removed

O estado é gerenciado automaticamente pelo sistema.

8. Políticas

Contrato Público

Toda capability deve possuir um contrato público bem definido.

Versionamento

Capabilities seguem versionamento semântico.

Mudanças incompatíveis exigem nova versão principal.

Descoberta

Plugins nunca devem assumir qual plugin fornece determinada capability.

A descoberta sempre ocorre através do Discovery Service.

Múltiplos Provedores

O sistema pode possuir diversas implementações para a mesma capability.

A seleção é feita pelo Capability Resolver.

Segurança

Invocações passam obrigatoriamente pelo Security Framework.

Falhas

Uma falha na execução de uma capability não deve comprometer o restante do sistema.

9. Integrações

O Capability System integra-se com:

- Plugin Manager;
- Plugin API;
- Plugin Context;
- Security Framework;
- Policy Engine;
- Event System;
- State Engine;
- Logging Framework;
- Scheduler;
- Resource Manager;
- Configuration System;
- Plugin Inspector.

Ele não depende diretamente de implementações dos plugins.

10. Métricas

O Capability System registra:

- número de capabilities registradas;
- número de provedores;
- invocações realizadas;
- tempo médio de execução;
- falhas por capability;
- falhas de permissão;
- capabilities mais utilizadas;
- versões em uso;
- quantidade de descobertas realizadas;
- taxa de sucesso das invocações.

Essas métricas auxiliam na otimização da plataforma e no diagnóstico de problemas.

11. Exemplos

Registro

Um plugin de música registra:

media.play

Outros plugins passam a descobrir essa capability.

Descoberta

Um plugin procura:

media.play

O Discovery Service retorna todos os provedores compatíveis.

Invocação

1. Plugin solicita "media.play".
2. O Resolver seleciona o provedor adequado.
3. O Invocation Engine encaminha a chamada.
4. O plugin provedor executa a ação.
5. O resultado é retornado ao solicitante.

Múltiplos Provedores

Dois players de música registram:

media.play

O Capability Resolver escolhe qual implementação utilizar com base nas políticas configuradas.

12. Regras Arquiteturais

- Plugins nunca invocam outros plugins diretamente.
- Toda comunicação ocorre por contratos públicos.
- O Capability System não executa lógica de negócio.
- Capacidades devem ser independentes de implementações específicas.
- Toda capability deve possuir identificador único e versão.
- Invocações devem ser preferencialmente assíncronas quando envolverem operações demoradas.

13. Limitações

O Capability System não é responsável por:

- gerenciar estados compartilhados;
- distribuir eventos;
- controlar ciclo de vida de plugins;
- armazenar dados;
- executar políticas de energia.

Essas funções pertencem a outros subsistemas.

14. Futuras Extensões

O Capability System poderá evoluir para suportar:

- descoberta baseada em contexto;
- balanceamento entre múltiplos provedores;
- composição automática de capabilities;
- cache inteligente de descobertas;
- negociação automática de versões;
- capacidades remotas entre dispositivos.

15. Resumo

O Capability System fornece o principal mecanismo de extensão funcional do Vertex. Ao permitir que plugins publiquem e consumam funcionalidades por meio de contratos públicos, ele elimina dependências diretas entre plugins, facilita a substituição de implementações e torna a plataforma altamente modular, escalável e compatível com futuras evoluções.